Creating a Training Loop for PyTorch Models

by **Adrian Tam** on April 8, 2023 in **Deep Learning with PyTorch** 0

PyTorch provides a lot of building blocks for a deep learning model, but a training loop is not part of them. It is a flexibility that allows you to do whatever you want during training, but some basic structure is universal across most use cases.

In this post, you will see how to make a training loop that provides essential information for your model training, with the option to allow any information to be displayed. After completing this post, you will know:

- The basic building block of a training loop
- How to use tqdm to display training progress

Kick-start your project with my book Deep Learning with PyTorch. It provides **self-study tutorials** with **working code**.

Let's get started.

Creating a training loop for PyTorch models
Photo by pat pat. Some rights reserved.

Overview

This post is in three parts; they are:

- Elements of Training a Deep Learning Model
- Collecting Statistics During Training
- Using tqdm to Report the Training Progress

Elements of Training a Deep Learning Model

As with all machine learning models, the model design specifies the algorithm to manipulate an input and produce an output. But in the model, there are parameters that you need to fine-tune to achieve that. These model parameters are also called the weights, biases, kernels, or other names depending on the particular model and layers. Training is to feed in the sample data to the model so that an optimizer can fine-tune these parameters.

When you train a model, you usually start with a dataset. Each dataset is a fairly large number of data samples. When you get a dataset, it is recommended to split it into two portions: the training set and the test set. The training set is further split into batches and used in the training loop to drive the gradient descent algorithms. The test set, however, is used as a benchmark to tell how good your model is. Usually, you do not use the training set as a metric but take the test set, which is not seen by the gradient descent algorithm, so you can tell if your model fits well to the unseen data.

Overfitting is when the model fits too well to the training set (i.e., at very high accuracy) but performs significantly worse in the test set. Underfitting is when the model cannot even fit well to the training set. Naturally, you don't want to see either on a good model.

Training of a neural network model is in epochs. Usually, one epoch means you run through the entire training set once, although you only feed one batch at a time. It is also customary to do some housekeeping tasks at the end of each epoch, such as benchmarking the partially trained model with the test set, checkpointing the model, deciding if you want to stop the training early, and collecting training statistics, and so on.

In each epoch, you feed data samples into the model in batches and run a gradient descent algorithm. This is one step in the training loop because you run the model in one forward pass (i.e., providing input and capturing output), and one backward pass (evaluating the loss metric from the output and deriving the gradient of each parameter all the way back to the input layer). The backward pass computes the gradient using automatic differentiation. Then, this gradient is used by the gradient descent algorithm to adjust the model parameters. There are multiple steps in one epoch.

Reusing the examples in a previous tutorial, you can download the dataset and split the dataset into two as follows:

```
import numpy as np
import torch

# load the dataset
dataset = np.loadtxt('pima-indians-diabetes.csv', delimiter=',')
X = dataset[:,0:8]
y = dataset[:,8]
X = torch.tensor(X, dtype=torch.float32)
y = torch.tensor(y, dtype=torch.float32).reshape(-1, 1)

# split the dataset into training and test sets
Xtrain = X[:700]
ytrain = y[:700]
Xtest = X[700:]
ytest = y[700:]
```

This dataset is small–only 768 samples. Here, it takes the first 700 as the training set and the rest as the test set.

It is not the focus of this post, but you can reuse the model, the loss function, and the optimizer from a previous post:

```
import torch.nn as nn
import torch.optim as optim

model = nn.Sequential(
    nn.Linear(8, 12),
    nn.ReLU(),
    nn.Linear(12, 8),
    nn.ReLU(),
    nn.Linear(8, 1),
    nn.Sigmoid()
)
print(model)

# loss function and optimizer
loss_fn = nn.BCELoss()  # binary cross entropy
optimizer = optim.Adam(model.parameters(), lr=0.001)
```

With the data and the model, this is the minimal training loop, with the forward and backward pass in each step:

```
n_epochs = 50    # number of epochs to run
batch_size = 10  # size of each batch
batches_per_epoch = len(Xtrain) // batch_size

for epoch in range(n_epochs):
    for i in range(batches_per_epoch):
        start = i * batch_size
        # take a batch
        Xbatch = Xtrain[start:start+batch_size]
        ybatch = ytrain[start:start+batch_size]
        # forward pass
        y_pred = model(Xbatch)
        loss = loss_fn(y_pred, ybatch)
        # backward pass
        optimizer.zero_grad()
        loss.backward()
        # update weights
        optimizer.step()
```

In the inner for-loop, you take each batch in the dataset and evaluate the loss. The loss is a PyTorch tensor that remembers how it comes up with its value. Then you zero out all gradients that the optimizer manages and call `loss.backward()` to run the backpropagation algorithm. The result sets up the gradients of all the tensors that the tensor `loss` depends on directly and indirectly. Afterward, upon calling `step()`, the optimizer will check each parameter that it manages and update them.

After everything is done, you can run the model with the test set to evaluate its performance. The evaluation can be based on a different function than the loss function. For example, this classification problem uses accuracy:

```
...

# evaluate trained model with test set
with torch.no_grad():
    y_pred = model(X)
accuracy = (y_pred.round() == y).float().mean()
print("Accuracy {:.2f}".format(accuracy * 100))
```

Putting everything together, this is the complete code:

```
import numpy as np
import torch
import torch.nn as nn
import torch.optim as optim

# load the dataset
dataset = np.loadtxt('pima-indians-diabetes.csv', delimiter=',')
X = dataset[:,0:8]
y = dataset[:,8]
X = torch.tensor(X, dtype=torch.float32)
y = torch.tensor(y, dtype=torch.float32).reshape(-1, 1)

# split the dataset into training and test sets
Xtrain = X[:700]
ytrain = y[:700]
Xtest = X[700:]
ytest = y[700:]

model = nn.Sequential(
    nn.Linear(8, 12),
    nn.ReLU(),
    nn.Linear(12, 8),
    nn.ReLU(),
    nn.Linear(8, 1),
    nn.Sigmoid()
)
print(model)

# loss function and optimizer
loss_fn = nn.BCELoss()  # binary cross entropy
optimizer = optim.Adam(model.parameters(), lr=0.001)

n_epochs = 50    # number of epochs to run
batch_size = 10  # size of each batch
batches_per_epoch = len(Xtrain) // batch_size

for epoch in range(n_epochs):
    for i in range(batches_per_epoch):
        start = i * batch_size
        # take a batch
        Xbatch = Xtrain[start:start+batch_size]
        ybatch = ytrain[start:start+batch_size]
        # forward pass
        y_pred = model(Xbatch)
        loss = loss_fn(y_pred, ybatch)
        # backward pass
        optimizer.zero_grad()
        loss.backward()
        # update weights
        optimizer.step()

# evaluate trained model with test set
with torch.no_grad():
    y_pred = model(X)
accuracy = (y_pred.round() == y).float().mean()
print("Accuracy {:.2f}".format(accuracy * 100))
```

Collecting Statistics During Training

The training loop above should work well with small models that can finish training in a few seconds. But for a larger model or a larger dataset, you will find that it takes significantly longer to train. While you're waiting for the training to complete, you may want to see how it's going as you may want to interrupt the training if any mistake is made.

Usually, during training, you would like to see the following:

- In each step, you would like to know the loss metrics, and you are expecting the loss to go down
- In each step, you would like to know other metrics, such as accuracy on the training set, that are of interest but not involved in the gradient descent

- At the end of each epoch, you would like to evaluate the partially-trained model with the test set and report the evaluation metric
- At the end of the training, you would like to be above to visualize the above metrics

These all are possible, but you need to add more code into the training loop, as follows:

```
n_epochs = 50    # number of epochs to run
batch_size = 10  # size of each batch
batches_per_epoch = len(Xtrain) // batch_size

# collect statistics
train_loss = []
train_acc = []
test_acc = []

for epoch in range(n_epochs):
    for i in range(batches_per_epoch):
        start = i * batch_size
        # take a batch
        Xbatch = Xtrain[start:start+batch_size]
        ybatch = ytrain[start:start+batch_size]
        # forward pass
        y_pred = model(Xbatch)
        loss = loss_fn(y_pred, ybatch)
        acc = (y_pred.round() == ybatch).float().mean()
        # store metrics
        train_loss.append(float(loss))
        train_acc.append(float(acc))
        # backward pass
        optimizer.zero_grad()
        loss.backward()
        # update weights
        optimizer.step()
        # print progress
        print(f"epoch {epoch} step {i} loss {loss} accuracy {acc}")
    # evaluate model at end of epoch
    y_pred = model(Xtest)
    acc = (y_pred.round() == ytest).float().mean()
    test_acc.append(float(acc))
    print(f"End of {epoch}, accuracy {acc}")
```

As you collect the loss and accuracy in the list, you can plot them using matplotlib. But be careful that you collected training set statistics at each step, but the test set accuracy only at the end of the epoch. Thus you would like to show the **average accuracy** from the training loop in each epoch, so they are comparable to each other.

```
import matplotlib.pyplot as plt

# Plot the loss metrics, set the y-axis to start from 0
plt.plot(train_loss)
plt.xlabel("steps")
plt.ylabel("loss")
plt.ylim(0)
plt.show()

# plot the accuracy metrics
avg_train_acc = []
for i in range(n_epochs):
    start = i * batch_size
    average = sum(train_acc[start:start+batches_per_epoch]) / batches_per_epoch
    avg_train_acc.append(average)

plt.plot(avg_train_acc, label="train")
plt.plot(test_acc, label="test")
plt.xlabel("epochs")
plt.ylabel("accuracy")
plt.ylim(0)
plt.show()
```

Putting everything together, below is the complete code:

```
import numpy as np
import torch
import torch.nn as nn
import torch.optim as optim

# load the dataset
dataset = np.loadtxt('pima-indians-diabetes.csv', delimiter=',') # split into input (X) and output (y) variables
X = dataset[:,0:8]
y = dataset[:,8]
X = torch.tensor(X, dtype=torch.float32)
y = torch.tensor(y, dtype=torch.float32).reshape(-1, 1)

# split the dataset into training and test sets
Xtrain = X[:700]
ytrain = y[:700]
Xtest = X[700:]
ytest = y[700:]

model = nn.Sequential(
    nn.Linear(8, 12),
    nn.ReLU(),
    nn.Linear(12, 8),
    nn.ReLU(),
```

```
    nn.Linear(8, 1),
    nn.Sigmoid()
)
print(model)

# loss function and optimizer
loss_fn = nn.BCELoss()  # binary cross entropy
optimizer = optim.Adam(model.parameters(), lr=0.0001)

n_epochs = 50    # number of epochs to run
batch_size = 10  # size of each batch
batches_per_epoch = len(Xtrain) // batch_size

# collect statistics
train_loss = []
train_acc = []
test_acc = []

for epoch in range(n_epochs):
    for i in range(batches_per_epoch):
        # take a batch
        start = i * batch_size
        Xbatch = Xtrain[start:start+batch_size]
        ybatch = ytrain[start:start+batch_size]
        # forward pass
        y_pred = model(Xbatch)
        loss = loss_fn(y_pred, ybatch)
        acc = (y_pred.round() == ybatch).float().mean()
        # store metrics
        train_loss.append(float(loss))
        train_acc.append(float(acc))
        # backward pass
        optimizer.zero_grad()
        loss.backward()
        # update weights
        optimizer.step()
        # print progress
        print(f"epoch {epoch} step {i} loss {loss} accuracy {acc}")
    # evaluate model at end of epoch
    y_pred = model(Xtest)
    acc = (y_pred.round() == ytest).float().mean()
    test_acc.append(float(acc))
    print(f"End of {epoch}, accuracy {acc}")

import matplotlib.pyplot as plt

# Plot the loss metrics
plt.plot(train_loss)
plt.xlabel("steps")
plt.ylabel("loss")
plt.ylim(0)
plt.show()

# plot the accuracy metrics
avg_train_acc = []
for i in range(n_epochs):
    start = i * batch_size
    average = sum(train_acc[start:start+batches_per_epoch]) / batches_per_epoch
    avg_train_acc.append(average)

plt.plot(avg_train_acc, label="train")
plt.plot(test_acc, label="test")
plt.xlabel("epochs")
plt.ylabel("accuracy")
plt.ylim(0)
plt.show()
```

The story does not end here. Indeed, you can add more code to the training loop, especially in dealing with a more complex model. One example is checkpointing. You may want to save your model (e.g., using pickle) so that, if for any reason, your program stops, you can restart the training loop from the middle. Another example is early stopping, which lets you monitor the accuracy you obtained with the test set at the end of each epoch and interrupt the training if you don't see the model improving for a while. This is because you probably can't go further, given the design of the model, and you do not want to overfit.

Using tqdm to Report the Training Progress

If you run the above code, you will find that there are a lot of lines printed on the screen while the training loop is running. Your screen may be cluttered. And you may also want to see an animated progress bar to better tell you how far you are in the training progress. The library `tqdm` is the popular tool for creating the progress bar. Converting the above code to use tqdm cannot be easier:

```
for epoch in range(n_epochs):
    with tqdm.trange(batches_per_epoch, unit="batch", mininterval=0) as bar:
        bar.set_description(f"Epoch {epoch}")
        for i in bar:
            # take a batch
            start = i * batch_size
            Xbatch = Xtrain[start:start+batch_size]
            ybatch = ytrain[start:start+batch_size]
            # forward pass
            y_pred = model(Xbatch)
            loss = loss_fn(y_pred, ybatch)
            acc = (y_pred.round() == ybatch).float().mean()
            # store metrics
            train_loss.append(float(loss))
            train_acc.append(float(acc))
            # backward pass
            optimizer.zero_grad()
            loss.backward()
            # update weights
            optimizer.step()
            # print progress
            bar.set_postfix(
                loss=float(loss),
                acc=f"{float(acc)*100:.2f}%"
            )
    # evaluate model at end of epoch
    y_pred = model(Xtest)
    acc = (y_pred.round() == ytest).float().mean()
    test_acc.append(float(acc))
    print(f"End of {epoch}, accuracy {acc}")
```

The usage of `tqdm` creates an iterator using `trange()` just like Python's `range()` function, and you can read the number in a loop. You can access the progress bar by updating its description or "postfix" data, but you have to do that before it exhausts its content. The `set_postfix()` function is powerful as it can show you anything.

In fact, there is a `tqdm()` function besides `trange()` that iterates over an existing list. You may find it easier to use, and you can rewrite the above loop as follows:

```
starts = [i*batch_size for i in range(batches_per_epoch)]

for epoch in range(n_epochs):
    with tqdm.tqdm(starts, unit="batch", mininterval=0) as bar:
        bar.set_description(f"Epoch {epoch}")
        for start in bar:
            # take a batch
            Xbatch = Xtrain[start:start+batch_size]
            ybatch = ytrain[start:start+batch_size]
            # forward pass
            y_pred = model(Xbatch)
            loss = loss_fn(y_pred, ybatch)
            acc = (y_pred.round() == ybatch).float().mean()
            # store metrics
            train_loss.append(float(loss))
            train_acc.append(float(acc))
            # backward pass
            optimizer.zero_grad()
            loss.backward()
            # update weights
            optimizer.step()
            # print progress
            bar.set_postfix(
                loss=float(loss),
                acc=f"{float(acc)*100:.2f}%"
            )
    # evaluate model at end of epoch
    y_pred = model(Xtest)
    acc = (y_pred.round() == ytest).float().mean()
    test_acc.append(float(acc))
    print(f"End of {epoch}, accuracy {acc}")
```

The following is the complete code (without the matplotlib plotting):

```
import numpy as np
import torch
import torch.nn as nn
import torch.optim as optim
import tqdm

# load the dataset
dataset = np.loadtxt('pima-indians-diabetes.csv', delimiter=',') # split into input (X) and output (y) variables
X = dataset[:,0:8]
y = dataset[:,8]
X = torch.tensor(X, dtype=torch.float32)
y = torch.tensor(y, dtype=torch.float32).reshape(-1, 1)

# split the dataset into training and test sets
Xtrain = X[:700]
```

```
ytrain = y[:700]
Xtest = X[700:]
ytest = y[700:]

model = nn.Sequential(
    nn.Linear(8, 12),
    nn.ReLU(),
    nn.Linear(12, 8),
    nn.ReLU(),
    nn.Linear(8, 1),
    nn.Sigmoid()
)
print(model)

# loss function and optimizer
loss_fn = nn.BCELoss()  # binary cross entropy
optimizer = optim.Adam(model.parameters(), lr=0.0001)

n_epochs = 50    # number of epochs to run
batch_size = 10  # size of each batch
batches_per_epoch = len(Xtrain) // batch_size

# collect statistics
train_loss = []
train_acc = []
test_acc = []

for epoch in range(n_epochs):
    with tqdm.trange(batches_per_epoch, unit="batch", mininterval=0) as bar:
        bar.set_description(f"Epoch {epoch}")
        for i in bar:
            # take a batch
            start = i * batch_size
            Xbatch = Xtrain[start:start+batch_size]
            ybatch = ytrain[start:start+batch_size]
            # forward pass
            y_pred = model(Xbatch)
            loss = loss_fn(y_pred, ybatch)
            acc = (y_pred.round() == ybatch).float().mean()
            # store metrics
            train_loss.append(float(loss))
            train_acc.append(float(acc))
            # backward pass
            optimizer.zero_grad()
            loss.backward()
            # update weights
            optimizer.step()
            # print progress
            bar.set_postfix(
                loss=float(loss),
                acc=f"{float(acc)*100:.2f}%"
            )
    # evaluate model at end of epoch
    y_pred = model(Xtest)
    acc = (y_pred.round() == ytest).float().mean()
    test_acc.append(float(acc))
    print(f"End of {epoch}, accuracy {acc}")
```

Summary

In this post, you looked in detail at how to properly set up a training loop for a PyTorch model. In particular, you saw:

- What are the elements needed to implement in a training loop
- How a training loop connects the training data to the gradient descent optimizer
- How to collect information in the training loop and display them

More On This Topic

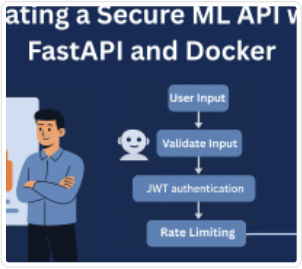
Creating Custom Layers and Loss Functions in PyTorch

Creating Powerful Ensemble Models with PyCaret

Take Control By Creating Targeted Lists of Machine…

Creating a PowerPoint Presentation using ChatGPT

Stable Diffusion Project: Creating Illustration



Creating a Secure Machine Learning API with FastAPI…

About Adrian Tam

Adrian Tam, PhD is a data scientist and software engineer.
View all posts by Adrian Tam →

‹ Develop Your First Neural Network with PyTorch, Step by Step
How to Evaluate the Performance of PyTorch Models ›

No comments yet.

Leave a Reply

Name (required)

Email (will not be published) (required)

SUBMIT COMMENT

Welcome!
I'm *Jason Brownlee* PhD
and I **help developers** get results with **machine learning**.
Read more

Never miss a tutorial:

Picked for you:


PyTorch Tutorial: How to Develop Deep Learning Models with Python


LSTM for Time Series Prediction in PyTorch


Deep Learning with PyTorch (9-Day Mini-Course)


Calculating Derivatives in PyTorch


Develop Your First Neural Network with PyTorch, Step by Step

Loving the Tutorials?

The Deep Learning with PyTorch EBook
is where you'll find the ***Really Good*** stuff.

>> SEE WHAT'S INSIDE

Machine Learning Mastery is part of Guiding Tech Media, a leading digital media publisher focused on helping people figure out technology. Visit our corporate website to learn more about our mission and team.